FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Keisha Belle Calibjo
Section:	AN43
Professor:	Creisola G. Tan
Date:	11/03/26

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice

- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule

3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?

- RBR was easier because each rule directly connects a condition to an action. The inference engine only needs to check each rule against the current facts.

2. Which is more flexible in solving unstructured problems?

- CBR is more flexible when a problem is difficult to describe with fixed rules but similar past cases are available. It can retrieve and adapt an earlier solution. Its usefulness depends on having relevant cases in the case base.

3. Which is better for situations with continuously changing data?

- CBR can adapt as new situations and solutions are retained in its case base. RBR can also respond to changing inputs each time it runs, but its rules stay fixed until someone updates them. For this activity, CBR offers more adaptation to new examples; RBR is useful when the decision conditions remain clear and stable.

4. Examples of real-world applications for both.
- RBR can support thermostat automation or rule-based decision support. CBR can support technical troubleshooting or customer service by finding solutions from similar past cases.

Your Code - Laboratory Tasks ⇩

```
"""TA 4: Knowledge Representation, Rule-Based Reasoning, and Case-Based
Reasoning.

Scenario: simple indoor comfort and rainy-day advice for one room.
Run normally to enter current room facts and revise the retrieved CBR
solution.
Run with --demo to reproduce the example output included in the report.
"""
from __future__ import annotations

import argparse
from copy import deepcopy
from typing import Any


# Knowledge representation: five facts, grouped into three knowledge
categories.
KNOWLEDGE_CATEGORIES = {
    "ambient_conditions": ("temperature_c", "is_raining", "humidity_pct"),
    "occupancy": ("room_occupied",),
    "ventilation": ("window_open",),
}

DEFAULT_FACTS: dict[str, Any] = {
    "temperature_c": 32,
    "is_raining": True,
    "humidity_pct": 85,
    "room_occupied": True,
    "window_open": False,
}
```

```python
# Rule functions return one action when their IF condition matches, else None.
def rule_hot_room(facts: dict[str, Any]) -> str | None:
        """IF the room is hot and occupied, THEN turn on the air conditioner."""
    if facts["temperature_c"] >= 30 and facts["room_occupied"]:
        return "Turn on the air conditioner"
    return None

def rule_rain(facts: dict[str, Any]) -> str | None:
    """IF it is raining, THEN bring an umbrella."""
    if facts["is_raining"]:
        return "Bring an umbrella"
    return None


def rule_humid_room(facts: dict[str, Any]) -> str | None:
        """IF humidity is high and the window is closed, THEN use a dehumidifier."""
    if facts["humidity_pct"] >= 70 and not facts["window_open"]:
        return "Use a dehumidifier"
    return None


RULES = (rule_hot_room, rule_rain, rule_humid_room)


def infer_actions(facts: dict[str, Any]) -> list[str]:
    """Inference engine: check every rule against working-memory facts."""
    actions: list[str] = []
    for rule in RULES:
        action = rule(facts)
        if action is not None:
            actions.append(action)
    return actions
```

```python
# Three retained past cases. Each case maps a problem profile to a solution.
CASE_BASE: list[dict[str, Any]] = [
    {
        "id": "case_1_hot_and_rainy",
        "problem": {"hot": True, "raining": True, "humid": False},
        "solution": "Bring an umbrella and use the air conditioner",
    },
    {
        "id": "case_2_hot_and_dry",
        "problem": {"hot": True, "raining": False, "humid": False},
        "solution": "Use a fan and drink water",
    },
    {
        "id": "case_3_cool_rainy_and_humid",
        "problem": {"hot": False, "raining": True, "humid": True},
        "solution": "Bring an umbrella and use a dehumidifier",
    },
]

def problem_profile(facts: dict[str, Any]) -> dict[str, bool]:
    """Convert detailed facts to comparable case features."""
    return {
        "hot": facts["temperature_c"] >= 30,
        "raining": facts["is_raining"],
        "humid": facts["humidity_pct"] >= 70,
    }


def similarity(query: dict[str, bool], past: dict[str, bool]) -> float:
    """Return the proportion of matching Boolean features (0.0 to 1.0)."""
    matches = sum(query[key] == past[key] for key in query)
    return matches / len(query)

def retrieve_case(
    facts: dict[str, Any], cases: list[dict[str, Any]]
) -> tuple[dict[str, Any], float]:
```

```
    """Retrieve the most similar case; stable ordering resolves ties."""
    if not cases:
        raise ValueError("The case base is empty; add a past case before
retrieval.")
    query = problem_profile(facts)
    best_case = max(cases, key=lambda case: similarity(query,
case["problem"]))
    score = similarity(query, best_case["problem"])
    return best_case, score

def get_facts(interactive: bool) -> dict[str, Any]:
    """Read a new situation, using the documented sample as each
default."""
    facts = deepcopy(DEFAULT_FACTS)
    if not interactive:
        return facts
    print("Enter current facts (press Enter to use the sample value).")
    raw = input(f"Temperature in C [{facts['temperature_c']}]: ").strip()
    if raw:
        facts["temperature_c"] = float(raw)
    raw = input(f"Is it raining? y/n [{'y' if facts['is_raining'] else
'n'}]: ").strip().lower()
    if raw:
        if raw not in {"y", "n"}:
            raise ValueError("Please enter y or n for raining.")
        facts["is_raining"] = raw == "y"
    raw = input(f"Humidity percent [{facts['humidity_pct']}]: ").strip()
    if raw:
        facts["humidity_pct"] = int(raw)
        raw = input(f"Is the room occupied? y/n [{'y' if
facts['room_occupied'] else 'n'}]: ").strip().lower()
    if raw:
        if raw not in {"y", "n"}:
            raise ValueError("Please enter y or n for room occupied.")
        facts["room_occupied"] = raw == "y"
    raw = input(f"Is a window open? y/n [{'y' if facts['window_open'] else
'n'}]: ").strip().lower()
```

```
    if raw:
        if raw not in {"y", "n"}:
            raise ValueError("Please enter y or n for window open.")
        facts["window_open"] = raw == "y"
    if not 0 <= facts["humidity_pct"] <= 100:
        raise ValueError("Humidity must be between 0 and 100 percent.")
    return facts

def cbr_cycle(
    facts: dict[str, Any], cases: list[dict[str, Any]], interactive: bool
) -> tuple[str, float, dict[str, Any]]:
    """Perform Retrieve, Reuse, Revise, Retain and return the retained
case."""
    selected, score = retrieve_case(facts, cases)  # Retrieve
    solution = selected["solution"]  # Reuse
    print(f"Retrieved: {selected['id']} (similarity {score:.0%})")
    print(f"Reused solution: {solution}")
    if interactive and input("Revise this solution? y/n [n]:
").strip().lower() == "y":
        revised = input("Enter the revised solution: ").strip()
        if revised:
            solution = revised  # Revise
    retained = {
        "id": f"case_{len(cases) + 1}_new_situation",
        "problem": problem_profile(facts),
        "solution": solution,
    }
    cases.append(retained)  # Retain
    print(f"Retained new case: {retained['id']}")
    return solution, score, retained

def print_knowledge_base(facts: dict[str, Any]) -> None:
    print("Knowledge Base (working memory):")
    for category, keys in KNOWLEDGE_CATEGORIES.items():
        values = ", ".join(f"{key}={facts[key]}" for key in keys)
        print(f"  {category}: {values}")
```

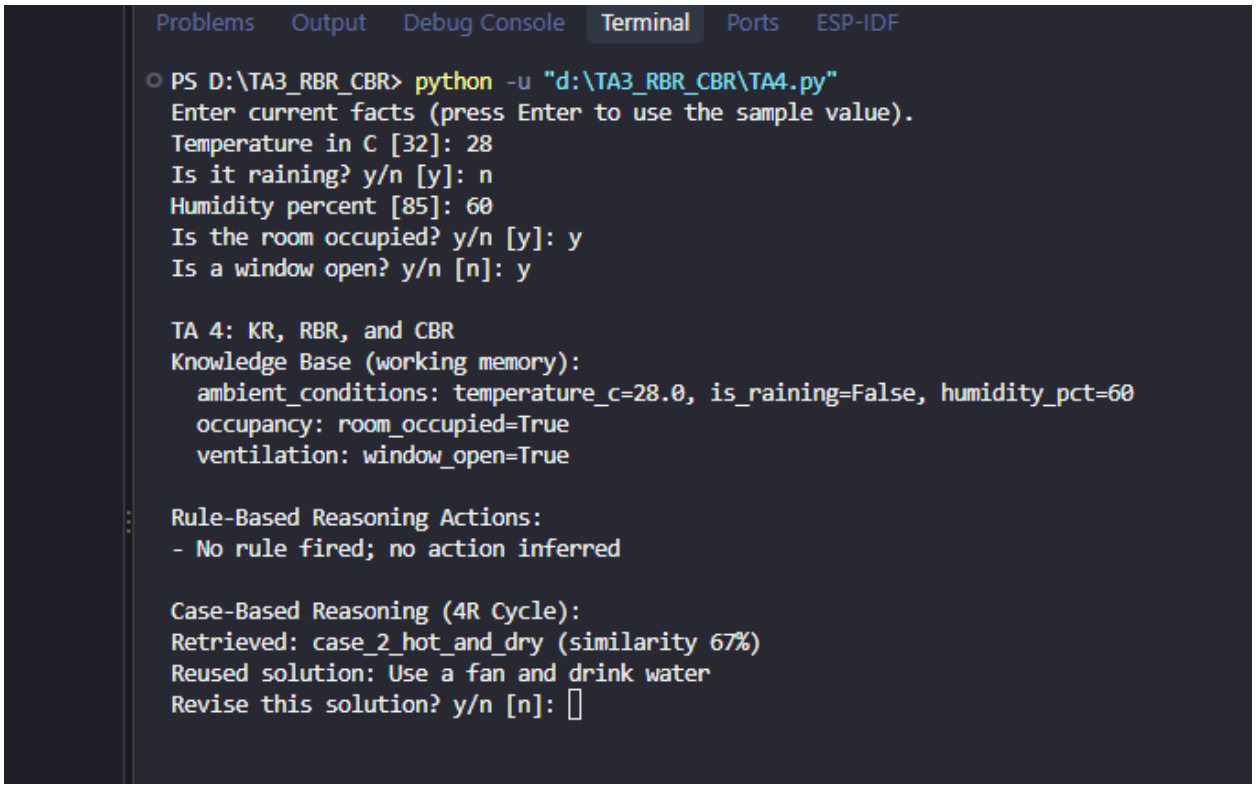
```
def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--demo", action="store_true", help="run the fixed
sample without prompts")
    args = parser.parse_args()
    facts = get_facts(interactive=not args.demo)

    print("\nTA 4: KR, RBR, and CBR")
    print_knowledge_base(facts)
    print("\nRule-Based Reasoning Actions:")
    actions = infer_actions(facts)
    if actions:
        for action in actions:
            print(f"- {action}")
    else:
        print("- No rule fired; no action inferred")

    print("\nCase-Based Reasoning (4R Cycle):")
        solution,  _,  _  =  cbr_cycle(facts,  CASE_BASE,  interactive=not
args.demo)
    print(f"Final CBR solution: {solution}")
    print(f"Case base size after retain: {len(CASE_BASE)}")


if __name__ == "__main__":
    main()
```

OUTPUT:


Problems
Output
Debug Console
Terminal
Ports
ESP-IDF
PS D:\TA3_RBR_CBR> python -u "d:\TA3_RBR_CBR\TA4.py"
Enter current facts (press Enter to use the sample value).
Temperature in C [32]: 28
Is it raining? y/n [y]: n
Humidity percent [85]: 60
Is the room occupied? y/n [y]: y
Is a window open? y/n [n]: y
TA 4: KR, RBR, and CBR
Knowledge Base (working memory):
ambient_conditions: temperature_c=28.0, is_raining=False, humidity_pct=60
occupancy: room_occupied=True
ventilation: window_open=True
Rule-Based Reasoning Actions:
- No rule fired; no action inferred
Case-Based Reasoning (4R Cycle):
Retrieved: case_2_hot_and_dry (similarity 67%)
Reused solution: Use a fan and drink water
Revise this solution? y/n [n]: